Information Sciences 730 (2026) 122875

Contents lists available at ScienceDirect

Information Sciences

journal homepage: www.elsevier.com/locate/ins

A verifiable privacy-preserving federated learning scheme based on homomorphic proxy re-encryption

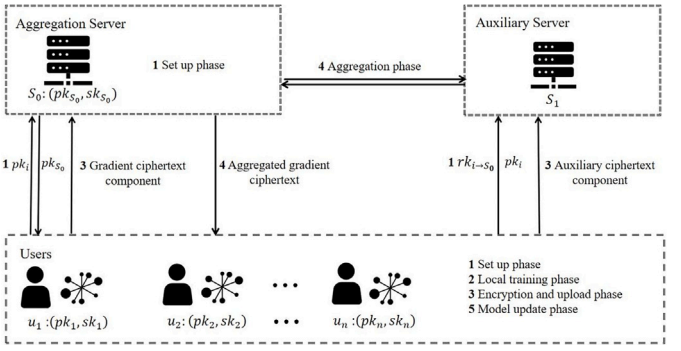
Zeyu Song [a], Zhenjie Huang [a, b, *], Yiping Cai [b]

[a] School of Mathematics and Statistics, Minnan Normal University, Zhangzhou, 363000, Fujian, China
[b] Fujian Key Laboratory of Granular Computing and Application, Minnan Normal University, Zhangzhou, 363000, Fujian, China

HIGHLIGHTS

- Propose a new verifiable privacy-preserving federated learning scheme (PPFL) based on homomorphic proxy re-encryption (H-PRE). The proposed scheme only requires two non-colluding servers to be semi-trusted, whereas previous schemes require at least one server to be fully trusted.
- Present a new approach for embedding homomorphic proxy re-encryption into a privacy-preserving federated learning scheme. Transmitting the two components of the homomorphic proxy re-encryption ciphertext to two different servers instead of sending the complete ciphertext to one server as in existing schemes, thus preventing a single server from compromising privacy.
- Analyze an existing privacy-preserving federated learning scheme and show that the curious server can collude with any user to compute other users' gradients, indicating that the scheme fails to achieve privacy.

GRAPHICAL ABSTRACT



We propose a new verifiable privacy-preserving federated learning scheme based on homomorphic proxy re-encryption. The proposed scheme requires only two non-colluding servers to be semi-trusted, whereas previous schemes require at least one server to be fully trusted.

ARTICLE INFO

Keywords:
Machine learning
Federated learning
Privacy-preserving
Homomorphic proxy re-encryption
Verifiable

ABSTRACT

Privacy-preserving federated learning (PPFL) enables multiple participants to train a global model using their local private data while preventing local gradients from leaking. We analyze a previous PPFL scheme and show that the curious server can collude with any participant to compute other participants' gradients, indicating the scheme fails to preserve privacy. To address the

* Corresponding author at: School of Mathematics and Statistics, Minnan Normal University, Zhangzhou, 363000, Fujian, China.
Email address: zjhuang@mnnu.edu.cn (Z. Huang).

https://doi.org/10.1016/j.ins.2025.122875
Received 16 July 2025; Received in revised form 6 November 2025; Accepted 6 November 2025
Available online 11 November 2025
0020-0255/© 2025 Elsevier Inc. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

shortcomings of previous similar schemes in privacy, trust, and verifiability, we present a new approach for embedding homomorphic proxy re-encryption (H-PRE) into a federated learning scheme and then propose a new PPFL scheme. The scheme transmits the two components of the H-PRE ciphertext to two different servers, rather than sending the entire ciphertext to a single server as in previous schemes. The new approach prevents a single server from acting maliciously and compromising privacy, reduces overhead, and improves efficiency. We also use a digital signature and linear homomorphic hash function to provide verifiability. The proposed scheme only requires that the two servers be semi-trusted, making it more practical for real-world applications. Analysis and simulation show that the scheme performs excellently in terms of computational and communication overhead, demonstrating its practicality.

1. Introduction

Machine learning (ML) [1] is one of the most critical technologies in artificial intelligence, utilizing large amounts of data to train complex models. However, in practical applications, data owners often hesitate to share data due to concerns about sensitive information leakage, leading to data silos and hindering innovation. In 2016, Google proposed a Federated Learning (FL) framework to address the issues of data silos and privacy protection [2]. In FL, users train a local model on their local dataset and then upload only the model gradients to a central server. The server aggregates the gradients from all users into a new global gradient, and sends it back to each user as the initial global gradient for the next round, continuing until the model converges. FL shares only the gradients or parameters of the users, without sharing the users' datasets [3]. This method protects data privacy but still faces risks of sensitive information leakage and other security threats. Mothukuri et al. [4] pointed out that the gradients or model parameters uploaded by users still contain sensitive information related to the user's data. Consequently, malicious attackers may be able to exploit these gradients or parameters to infer confidential information.

To prevent privacy leakage, researchers have proposed various methods to achieve PPFL, such as using Functional Encryption (FE) [5], Secure Multi-party Computation (SMPC) [6], Secret Sharing (SS) [7], Differential Privacy (DP) [8], Homomorphic Encryption (HE) [9], and Homomorphic Proxy Re-encryption (H-PRE) [10]. These techniques each have their own advantages and disadvantages: FE enables computational operations on gradients while maintaining encrypted state, but its high computational complexity makes it impractical for real-world applications. SMPC achieves collaborative computation of private data through cryptographic protocols or key sharing. However, each training round requires extensive synchronous interactions, with communication volume expanding quadratically with the number of users. In secret-sharing-based PPFL, users split their local gradients into shares, encrypt them, and send them to all other users. This process requires a significant amount of encryption and decryption operations, as well as high communication overhead. DP adds noise to the data to provide privacy protection. It is relatively easy to implement. However, the noise inevitably leads to a reduction in model accuracy. Homomorphic Encryption supports homomorphic operations between encrypted gradients, but it requires that the gradients being processed must be encrypted with the same public key. Since the holder of the public key possesses decryption capabilities, additional techniques must be combined to ensure the privacy of the gradients. In contrast, H-PRE combines the advantages of both HE and proxy re-encryption, ensuring gradient privacy without requiring additional technologies. This method significantly enhances the efficiency, flexibility, and scalability of FL systems while maintaining end-to-end privacy, providing an effective implementation approach for distributed collaborative machine learning.

Rodriguez et al. [11] discussed various threats and attacks in federated learning. They pointed out that during federated learning, a malicious server might aggregate gradients from only a portion of the users or even return random values to save computational resources and storage space. This could decrease the accuracy of the global gradient and may even prevent it from converging. Many schemes utilize different techniques to provide verifiability to ensure that users receive the correct aggregated gradients [7,12].

1.1. Motivation and contributions

Employing homomorphic proxy re-encryption (H-PRE) technology for privacy protection in federated learning is an innovative strategy. To the best of the authors' knowledge, there are currently only four H-PRE-PPFL schemes. Shen et al. [13] proposed a PPFL scheme based on H-PRE. By introducing fog nodes as proxies, the scheme converts users' ciphertexts into the aggregation server's ciphertexts, thereby preventing privacy leakage caused by collusion between individual users and the aggregation server. Li et al. [10] also utilized H-PRE technology to construct a PPFL scheme. The scheme employs a non-colluding dual-server model, where the server S_0, which is responsible for receiving gradient ciphertexts, re-encrypts the users' ciphertexts into the other server S_1's ciphertexts and aggregates the converted gradient ciphertexts. Fan et al. [14] proposed a multi-user ciphertext transformation framework adapted for FL. The cloud server can aggregate and distribute the global gradients without decrypting the local gradients of individual users. Bai et al. [15] provided a scheme that combines FL with identity-based homomorphic proxy re-encryption to resist privacy threats in intelligent connected vehicles, enabling efficient and secure collaborative training.

However, Shen et al. [13] involve multiple fog nodes responsible for ciphertext transformation. These nodes must not collude with the aggregation server, and at least one must be a trusted entity. Although Li et al. [10] claim that their servers can be semi-trusted, unfortunately, a curious server S_0 can collude with one user to obtain the local gradients of other users (see the analysis in Section 3 of this paper). Both [14,15] rely on a single server, which must be fully trusted rather than semi-trusted as they claim. A curious server could directly re-encrypt any user's gradient ciphertext and send it to a colluding user, allowing the colluding user to decrypt and

obtain the gradients of any other users. Semi-trusted means the server will honestly execute the protocol but may be curious about sensitive information and attack privacy and confidentiality. Additionally, none of the four existing schemes provides a verifiability mechanism for the aggregated gradients, which leaves them vulnerable to the security risks identified by Rodriguez et al. [11].

The above analysis shows that there is currently no verifiable and secure PPFL scheme under a semi-trusted server model. We propose a verifiable PPFL scheme under a non-colluding dual semi-trusted server model. The contributions of the paper are as follows:

- Analyze the PPFL scheme in [10] and show it fails to preserve privacy: a curious server S_0 can collude with any user u_{i^*} to obtain the gradient of any other user u_{j^*}.
- Propose a new verifiable PPFL scheme based on H-PRE. The proposed scheme only requires two servers that are semi-trusted and do not collude with each other. In contrast, existing schemes not only require that the servers do not collude but also demand that at least one of the servers is fully trusted.
- Present an innovative approach for embedding homomorphic proxy re-encryption (H-PRE) into a federated learning scheme and propose a new PPFL scheme. The scheme transmits the two components of the H-PRE ciphertext to two different servers, and each server independently aggregates one component of the ciphertext, thereby preventing a single server from compromising privacy. In contrast, existing schemes typically send the entire ciphertext (both components) to a single server, providing the server an opportunity to act maliciously.
- Employ a digital signature and linear homomorphic hash function (LHH) to provide verifiability. The server responsible for aggregating gradients can also verify the aggregated gradient, preventing deception by the auxiliary server. Existing similar schemes lack such a verifiability mechanism.
- Significantly reduce both computational and communication overhead. Analysis and simulation show that the scheme performs excellently in terms of computational and communication overhead, demonstrating its practicality.

1.2. Related work

Due to space limitations, we only review and analyze works closely related to the theme of this paper.

Privacy − Preserving Federated Learning Based on Homomorphic Encryption. Shi et al. [9] provided a PPFL scheme based on homomorphic encryption (HE) and secret sharing (SS), enabling secure and efficient model aggregation without sharing private keys, while also being resilient to collusion and node dropouts. Ma et al. [16] utilized multi-key homomorphic encryption to protect honest users' gradients, which can resist collusion by $n-2$ users. Cai et al. [17] developed a secure PPFL framework using fully HE and the SS, featuring a flexible offline protection mechanism. Du et al. [18] presented a lightweight PPFL to achieve collusion resistance, support dynamic participation, and enable efficient and secure aggregation without sharing secret keys. Zhao et al. [19] presented a lossless FL scheme using Paillier homomorphic encryption, effectively protecting sensitive data from inference attacks by the aggregation server and other participants. Shen et al. [20] presented an FL scheme resistant to server collusion, leveraging unidirectional H-PRE and verifiable secret sharing to protect gradient and random number privacy while achieving efficient aggregation and dynamic user support.

Verifiable Privacy − Preserving Federated Learning. Wang et al. [12] provided a robust and verifiable FL scheme, which utilized double masking and linear homomorphic hash to protect gradient privacy and achieved verifiable aggregation and robust fault tolerance. Xu et al. [7] presented an FL scheme named VerifyNet. The scheme used a double-masking protocol to protect the gradient privacy of users who dropped out or uploaded with delay and verified the aggregation results by sending multiple proofs to the server. Guo et al. [21] combined linear homomorphic hash function with commitment schemes to achieve efficient aggregation verification while supporting user dropout. Ren et al. [22] analyzed [7] and [21], discovering that a malicious server could forge hash values and aggregation results to pass user verification and thus used double-masking and homomorphic encryption to construct a PPFL scheme. Eltaras et al. [23] constructed a verifiable and efficient PPFL scheme using masking and homomorphic message authentication codes (MAC). However, this scheme has security vulnerabilities, allowing the server to forge aggregation gradients and MAC values to deceive users.

1.3. Organization

The remainder of the paper is organized as follows. In Section 2, we discuss the cryptographic primitives employed in this study. In Section 3, we analyze two problems of the Li et al. [10] scheme. We present an overview of our proposed scheme in Section 4. Section 5 is dedicated to a thorough security analysis, and Section 6 conducts experimental comparisons and accuracy analysis. Finally, we conclude the paper in Section 7.

2. Preliminaries

2.1. Federated learning

Federated learning (FL) is a distributed machine learning approach that allows multiple users to cooperate in training a global model on their local private datasets. FL is a multi-round iterative process that continues until convergence. The t-th round of FL is described as follows:

(1) **Local training**: Each user receives the global gradient $\boldsymbol{w}_{t-1}$ from the server, then trains on their local dataset to obtain the local gradient $\boldsymbol{w}_{t,i}$ for the t-th round.

(2) **Data upload**: After completing local training, each user uploads $\boldsymbol{w}_{t,i}$ to the server.
(3) **Aggregation**: The server aggregates the local gradients to obtain the new global gradient $\boldsymbol{w}_t$, and sends it to each user as the global gradient for the next round.

2.2. Homomorphic proxy re-encryption

Li et al. [10] utilized an improved H-PRE scheme [24]:

Setup (1^λ): Input a security parameter λ, generate a prime number p of length λ, two multiplicative cyclic groups $\mathbb{G}$, $\mathbb{G}_T$ with the same order p and a bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$. Choose a generator g of group $\mathbb{G}$, compute $Z = e(g, g)$. Output the system public parameters $pp = (p, \mathbb{G}, \mathbb{G}_T, e, g, Z)$.

KeyGen (pp): Inputting pp, u_i chooses $v_{i1}, v_{i2} \in \mathbb{Z}_p$, sets $sk_i = (sk_{i1}.sk_{i2}) = (v_{i1}, v_{i2})$, and computes $pk_i = (pk_{i1}.pk_{i2}) = (g^{v_{i1}}, Z^{v_{i2}})$.

Re-KeyGen (sk_i, pk_j): Inputting pp, u_i uses sk_i and pk_j of u_j to compute the re-encryption key $rk_{i\to j} = pk_{j1}^{v_{i2}} = g^{v_{j1}v_{i2}} \in \mathbb{G}$.

First-Level Encryption (pp, pk_i, m): Input pp, pk_i and the message m, choose $r \in_R \mathbb{Z}_p$, compute:

$$C_{i1}^1 = e(g, g^{v_{i1}})^r = Z^{v_{i1}r}, C_{i2}^1 = Z^{m+r}. \tag{1}$$

Output the first-level ciphertext:

$$[m]_{pk_i}^1 = (C_{i1}^1, C_{i2}^1) = (Z^{v_{i1}r}, Z^{m+r}). \tag{2}$$

Second-Level Encryption (pp, pk_i, m): Input pp, pk_i and the message m, choose $r \in_R \mathbb{Z}_p$, compute:

$$C_{i1}^2 = g^r, C_{i2}^2 = Z^m pk_{i2}^r = Z^{m+v_{i2}r}. \tag{3}$$

Output the second-level ciphertext:

$$[m]_{pk_i}^2 = (C_{i1}^2, C_{i2}^2) = (g^r, Z^{m+v_{i2}r}). \tag{4}$$

Re-Encryption $(pp, rk_{i\to j}, [m]_{pk_i}^2)$: Input pp, $rk_{i\to j}$ and $[m]_{pk_i}^2$, compute:

$$C_{ij}^1 = e(g^r, rk_{i\to j}) = e(g^r, g^{v_{j1}v_{i2}}) = Z^{v_{j1}v_{i2}r}, \tag{5}$$

Output the re-encrypted ciphertext (first-level ciphertext):

$$[m]_{pk_j}^1 = (Z^{v_{j1}v_{i2}r}, Z^{m+v_{i2}r}). \tag{6}$$

First-Level Decryption $(pp, sk_i, [m]_{pk_i}^1)$: Input pp, u_i's secret key sk_i and $[m]_{pk_i}^1$, compute:

$$Z^m = C_{i2}^1 \cdot ((C_{i1}^1)^{(v_{i1})^{-1}})^{-1}. \tag{7}$$

Second-Level Decryption $(pp, sk_i, [m]_{pk_i}^2)$: Input pp, sk_i and $[m]_{pk_i}^2$, compute:

$$Z^m = C_{i2}^2 \cdot (e(C_{i1}^2, g)^{v_{i2}})^{-1}. \tag{8}$$

To obtain the plaintext $m \in \mathbb{Z}_p$, it is necessary to solve the discrete logarithm for Z^m. Solving the discrete logarithm for larger integers of m is a computationally hard problem. However, when the length of m is less than 32 bits, m can be computed [25].

Both the first-level and second-level ciphertexts of this scheme have the following homomorphic properties,

$$\begin{aligned}
[m_1]_{pk_i}^1 \oplus [m_2]_{pk_i}^1 &= (Z^{v_{i1}r_1}, Z^{m_1+r_1}) \oplus (Z^{v_{i1}r_2}, Z^{m_2+r_2}) \\
&= (Z^{v_{i1}r_1} Z^{v_{i1}r_2}, Z^{m_1+r_1} Z^{m_2+r_2}) \\
&= (Z^{v_{i1}(r_1+r_2)}, Z^{(m_1+m_2)+(r_1+r_2)}) \\
&= [m_1 + m_2]_{pk_i}^1.
\end{aligned} \tag{9}$$

$$\begin{aligned}
[m_1]_{pk_i}^2 \oplus [m_2]_{pk_i}^2 &= (g^{r_1}, Z^{m_1+v_{i2}r_1}) \oplus (g^{r_2}, Z^{m_2+v_{i2}r_2}) \\
&= (g^{r_1} g^{r_2}, Z^{m_1+v_{i2}r_1} Z^{m_2+v_{i2}r_2}) \\
&= (g^{r_1+r_2}, Z^{(m_1+m_2)+v_{i2}(r_1+r_2)}) \\
&= [m_1 + m_2]_{pk_i}^2.
\end{aligned} \tag{10}$$

2.3. Linear homomorphic hash function

A Linear Homomorphic Hash Function scheme consists of three algorithms as follows:

Setup$(1^\lambda, 1^d)$: Input a security parameter λ and vector dimension d, generate a prime number p of length λ and a prime p-order group $\mathbb{G}$ with a generator g. Choose d different elements $g_1, g_2, \ldots, g_d$ in $\mathbb{G}$. Output the system public parameters $pp = (g, p, \mathbb{G}, g_1, g_2, \ldots, g_d)$.

LHH.Hash$(pp, \boldsymbol{x}, d)$: Input pp, a vector $\boldsymbol{x} = (x^1, x^2, \dots, x^d)$, compute linear homomorphic hash function value

$$H_{lhh}(\boldsymbol{x}) = \prod_{l=1}^{d} g_l^{x_l}. \tag{11}$$

LHH has the two properties:

(1) **Homomorphism**: For two vectors $\boldsymbol{x} = (x^1, x^2, \dots, x^d)$, $\boldsymbol{y} = (y^1, y^2, \dots, y^d)$,

$$\begin{aligned} H_{lhh}(\boldsymbol{x}+\boldsymbol{y}) &= \prod_{l=1}^{d} g_l^{x_l+y_l} \\ &= \prod_{l=1}^{d} g_l^{x_l} \prod_{l=1}^{d} g_l^{y_l} \\ &= H_{lhh}(\boldsymbol{x}) \cdot H_{lhh}(\boldsymbol{y}). \end{aligned} \tag{12}$$

(2) **Collision Resistance**: No probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ can find two distinct vectors $\boldsymbol{x}$, $\boldsymbol{y}$ such that $H_{lhh}(\boldsymbol{x}) = H_{lhh}(\boldsymbol{y})$.

2.4. BLS signature

The Boneh-Lynn-Shacham (BLS) signature scheme consists of four algorithms:

Setup (1^λ): Input a security parameter λ, generate a prime number p of length λ, two multiplicative cyclic groups $\mathbb{G}$, $\mathbb{G}_T$ with the same order p and a bilinear map $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$. Choose a generator g of group $\mathbb{G}$ and a secure hash function $H_{\mathbb{G}} : \{0,1\}^* \to \mathbb{G}$. Output the system public parameters $pp = (p, \mathbb{G}, \mathbb{G}_T, e, g, H_{\mathbb{G}})$.

KeyGen (pp): Input pp, choose $k \in_R \mathbb{Z}_p$, let the signature secret key be $ssk = k$, and compute the verification public key $spk = g^k$.

Sign (pp, ssk, m): Input pp, ssk and the message m, compute $h = H_{\mathbb{G}}(m)$, generate a signature $\sigma = h^k = H_{\mathbb{G}}(m)^k$.

Verify (pp, spk, σ, m): Input pp, spk, σ and the message m. Verify whether σ satisfies the equation $e(spk, h) = e(\sigma, g)$. If it does, output 1; otherwise, output 0.

2.5. Chinese remainder theorem

Let $m_1, m_2, \dots, m_d$ be pairwise coprime positive integers. For any positive integers $a_1, a_2, \dots, a_d$, the system of congruences

$$\begin{cases} x \equiv a_1 \pmod{m_1} \\ x \equiv a_2 \pmod{m_2} \\ \quad \vdots \\ x \equiv a_d \pmod{m_d} \end{cases} \tag{13}$$

has a unique solution $x \equiv \frac{M}{m_1} e_1 a_1 + \frac{M}{m_2} e_2 a_2 + \cdots + \frac{M}{m_d} e_d a_d$, where $M = \prod_{i=1}^{d} m_i$ and $e_i = \left(\frac{M}{m_i}\right)^{-1} \pmod{m_i}$.

The Chinese Remainder Theorem (CRT) can be used to pack d positive integers $a_1, a_2, \dots, a_d$ into a single larger integer x. Conversely, a large integer x can be unpacked back into the d positive integers $a_1, a_2, \dots, a_d$.

2.6. Hardness assumptions

Let $\mathbb{G}$, $\mathbb{G}_T$ be two multiplicative cyclic groups with the same prime order p. Let $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be a bilinear map, g be a generator of $\mathbb{G}$ and $a, b, c \in_R \mathbb{Z}_p^*$. We present two trivial variants of the classic Decisional Diffie-Hellman (DDH) assumption.

Decisional Diffie – Hellman (DDH – v1) Assumption (Variant 1).

Let $Y = (\mathbb{G}, \mathbb{G}_T, p, e, g, e(g,g)^a, g^b, g^{ac})$, $D = e(g,g)^{ab}$, $R \in_R \mathbb{G}_T$. Given a random instance (Y, T) satisfying $Pr[T = D] = Pr[T = R] = \frac{1}{2}$, distinguish whether $T = D$ or $T = R$.

DDH – v1 Assumption. The advantage of any probabilistic polynomial-time (PPT) adversary $\mathcal{A}$ in solving the DDH-v1 problem is negligible, that is,

$$Adv_{DDH-v1} = |Pr[\mathcal{A}(Y,T) = 1 \mid T = D] - Pr[\mathcal{A}(Y,T) = 1 \mid T = R]| \le \eta(\kappa),$$

where $\eta(\cdot)$ is a negligible function.

Decisional Diffie – Hellman (DDH – v2) Assumption (Variant 2).

Let $Y = (\mathbb{G}, \mathbb{G}_T, p, e, g, e(g,g)^a, g^b)$, $D = e(g,g)^{ab}$, $R \in_R \mathbb{G}_T$. Given a random instance (Y, T) satisfying $Pr[T = D] = Pr[T = R] = \frac{1}{2}$, distinguish whether $T = D$ or $T = R$.

DDH – v2 Assumption. The advantage of any PPT adversary $\mathcal{A}$ in solving the DDH-v2 problem is negligible, that is,

$$Adv_{DDH-v2} = |Pr[\mathcal{A}(Y,T) = 1 \mid T = D] - Pr[\mathcal{A}(Y,T) = 1 \mid T = R]| \le \eta(\kappa),$$

where $\eta(\cdot)$ is a negligible function.

3. Analysis of the Li et al.'s scheme

We analyze [10], highlighting that it fails to protect the local gradients of users.

3.1. Li et al.'s scheme review

Li et al. [10] employ a non-colluding dual semi-trusted servers model S_0 and S_1. The scheme has five stages: Initialization Phase, Local Training Phase, Data Upload Phase, Aggregation Phase, and Model Update Phase. It is assumed that there are n users, each possessing their own local dataset D_i. They cooperate in training a model with a d-dimensional gradient vector.

1. **Initialization Phase**:
   (1) u_i chooses $v_{i1}, v_{i2} \in_R \mathbb{Z}_p$ as its secret key $sk_i = (sk_{i1}.sk_{i2}) = (v_{i1}, v_{i2})$, and computes $pk_i = (pk_{i1}.pk_{i2}) = (g^{v_{i1}}, Z^{v_{i2}})$, and sends pk_i to S_1.
   (2) S_1 chooses $v_{S_11}, v_{S_12} \in_R \mathbb{Z}_p$ as its secret key $sk_{S_1} = (sk_{S_11}.sk_{S_12}) = (v_{S_11}, v_{S_12})$, and computes $pk_{S_1} = (pk_{S_11}.pk_{S_12}) = (g^{v_{S_11}}, Z^{v_{S_12}})$, and sends pk_{S_1} to all users.
   (3) u_i uses sk_i and pk_{S_1} to compute $rk_{i\to S_1}$, and transmits $rk_{i\to S_1}$ through a secure channel to S_0.
   (4) S_1 uses sk_{S_1} and pk_i to compute $rk_{S_1\to i}$, and transmits $rk_{S_1\to i}$ through a secure channel to S_0.
2. **Local Training Phase**: Each user trains using the current global gradient $\boldsymbol{w}$ and their local dataset D_i to generate the local gradient $\boldsymbol{w}_i$.
3. **Data Upload Phase**: Each user encrypts $\boldsymbol{w}_i$ with pk_i to get $[\boldsymbol{w}_i]^2_{pk_i}$, and sends it to S_0.
4. **Aggregation Phase**
   (1) Re-encryption: Receiving $[\boldsymbol{w}_i]^2_{pk_i}$, S_0 uses $rk_{i\to S_1}$ to re-encrypt $[\boldsymbol{w}_i]^2_{pk_i}$ into $[\boldsymbol{w}_i]^1_{pk_{S_1}}$.
   (2) Aggregation: S_0 aggregates gradient ciphertexts:

$$[\boldsymbol{w}_{agg}]^1_{pk_{S_1}} = \oplus^n_{i=1}[\boldsymbol{w}_i]^1_{pk_{S_1}}. \tag{14}$$

   (3) Transformation: S_0 and S_1 interactively execute the Secure Ciphertext Transformation Protocol (SCT) to convert $[\boldsymbol{w}_{agg}]^1_{pk_{S_1}}$ into $[\boldsymbol{w}_{agg}]^2_{pk_{S_1}}$. SCT is depicted in Fig. 1.
   (4) Re-encryption: S_0 re-encrypts $[\boldsymbol{w}_{agg}]^2_{pk_{S_1}}$ into $[\boldsymbol{w}_{agg}]^1_{pk_i}$ respectively, and sends it to the corresponding u_i.
5. **Model Update Phase**: Receiving $[\boldsymbol{w}_{agg}]^1_{pk_i}$, each user uses sk_i to decrypt $[\boldsymbol{w}_{agg}]^1_{pk_i}$ to get the aggregated gradient $\boldsymbol{w}_{agg}$, and computes $\frac{1}{n}\boldsymbol{w}_{agg}$ as the initial global gradient for the next round.

3.2. Analysis of Li et al.'s scheme

Li et al. [10] claimed that their scheme can protect users' gradients, ensuring no adversary can obtain individual user's gradient. Unfortunately, after careful and thorough analysis, we found that [10] fails to protect gradient privacy. A curious server S_0 can collude with a specific user u_{i^*} to obtain the gradients of any user u_{j^*}.

Secure Ciphertext Transformation Protocol

• S_0
-Chooses $a \in \mathbb{Z}_p$, lets a d-dimensional vector $\boldsymbol{\alpha} = (a, a, \ldots, a)$.
-Encrypts $\boldsymbol{\alpha}$ to get $[\boldsymbol{\alpha}]^1_{pk_{S_1}}$.
-Utilizes $[\boldsymbol{\alpha}]^1_{pk_{S_1}}$ to blind $[\boldsymbol{w}_{agg}]^1_{pk_{S_1}}$:

$$[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^1_{pk_{S_1}} = [\boldsymbol{w}_{agg}]^1_{pk} + [\boldsymbol{\alpha}]^1_{pk_{S_1}}.$$

-Sends $[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^1_{pk_{S_1}}$ to S_1.
• S_1
-Receiving $[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^1_{pk_{S_1}}$, decrypts it to get $\boldsymbol{w}_{agg} + \boldsymbol{\alpha}$.
-Encrypts $\boldsymbol{w}_{agg} + \boldsymbol{\alpha}$ as $[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^2_{pk_{S_1}}$.
-Sends $[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^2_{pk_{S_1}}$ to S_0.
• S_0
-Receiving $[\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^2_{pk_{S_1}}$, encrypts $\boldsymbol{\alpha}$ to get $[\boldsymbol{\alpha}]^2_{pk_{S_1}}$.
-Performs unmasking: $[\boldsymbol{w}_{agg}]^2_{pk_{S_1}} = [\boldsymbol{w}_{agg} + \boldsymbol{\alpha}]^2_{pk_{S_1}} - [\boldsymbol{\alpha}]^2_{pk_{S_1}}$.

Fig. 1. SCT protocol.

During the aggregation stage, a curious server S_0 aggregates only the ciphertexts of gradients from user u_{i^*} and user u_{j^*} instead of aggregating the ciphertexts of gradients from all users.

$$\left[\boldsymbol{w}^*\right]^1_{pk_{S_1}} = \left[\boldsymbol{w}_{i^*}\right]^1_{pk_{S_1}} + \left[\boldsymbol{w}_{j^*}\right]^1_{pk_{S_1}} = \left[\boldsymbol{w}_{i^*} + \boldsymbol{w}_{j^*}\right]^1_{pk_{S_1}}. \tag{15}$$

S_0 submits $[\boldsymbol{w}^*]^1_{pk_{S_1}}$ to interact with S_1 to execute the SCT protocol. Upon completion of the protocol, S_0 obtains $[\boldsymbol{w}^*]^2_{pk_{S_1}}$ and re-encrypts $[\boldsymbol{w}^*]^2_{pk_{S_1}}$ into $[\boldsymbol{w}^*]^1_{pk_{i^*}}$. S_0 sends $[\boldsymbol{w}^*]^1_{pk_{i^*}}$ to u_{i^*}.

Receiving $[\boldsymbol{w}^*]^1_{pk_{i^*}}$, u_{i^*} decrypts $[\boldsymbol{w}^*]^1_{pk_{i^*}}$ to get $\boldsymbol{w}^* = \boldsymbol{w}_{i^*} + \boldsymbol{w}_{j^*}$. By subtracting its gradient $\boldsymbol{w}_{i^*}$ from $\boldsymbol{w}^*$, u_{i^*} can obtain the gradient $\boldsymbol{w}_{j^*}$ of u_{j^*}. It is evident that [10] fails to achieve privacy protection.

4. Our scheme

We provide a verifiable PPFL scheme under a non-colluding dual semi-trusted server model.

4.1. Design concept

The design concept of our scheme is as follows:

(1) Gradient Ciphertext Distribution and Aggregation: Each user divides their local gradient ciphertext into a gradient ciphertext component and an auxiliary ciphertext component. These components are transmitted to different servers. Each server aggregates one of the ciphertext components respectively. This prevents a single server from acting maliciously, allowing both servers to be only semi-trusted. This approach enhances security and reduces computational and communication overhead.
(2) Verifiability: The scheme employs digital signatures and linear homomorphic hash function to provide verifiability. The server responsible for aggregating gradients can verify the aggregated gradient to prevent cheating by the auxiliary server.
(3) Data Integrity and Authentication: All transmitted data are accompanied by digital signatures to provide source authentication and integrity verification. This ensures the correct set of online users and the correctly aggregated gradients.
(4) Minimizing Inter-Server Interaction for Efficiency: The scheme aims to reduce interaction between servers to improve efficiency. Interaction is only required once, specifically when determining the final set of online users.

4.2. System model

Our scheme adopts a non-colluding dual semi-trusted server model. The system consists of 2 non-colluding semi-trusted servers S_0, S_1 and n users. The functionalities of each entity are as follows:

User u_i: During the training phase, u_i receives the global gradient $\boldsymbol{w}_{t-1}$ from S_0, then trains on their local dataset to compute the local gradient $\boldsymbol{w}_i$. In the encryption and upload phase, u_i encrypts the local gradient to obtain the gradient ciphertext. The ciphertext is divided into two parts: the part containing the gradient information is called the gradient ciphertext component, and the part not containing gradient information is called the auxiliary ciphertext component. u_i uploads the gradient ciphertext component along with the hash value of the local gradient and the corresponding signature to S_0. u_i also uploads the auxiliary ciphertext component and the corresponding signature to S_1. In the verification and update phase, u_i decrypts and verifies the aggregated gradient. Once verified, u_i updates the global gradient.

Aggregation Server S_0: S_0 acts as the aggregator of the gradient ciphertexts. First, it interacts with S_1 to determine the set of online users and get the aggregated auxiliary ciphertext component from S_1. Second, S_0 aggregates the gradient ciphertext components received from the online users, and combines the aggregated gradient ciphertext component and the aggregated additional ciphertext component into the complete ciphertext. S_0 decrypts the complete ciphertext to obtain the aggregated gradient, then verifies the correctness of the aggregated gradient. Finally, S_0 encrypts and signs the aggregated gradient, subsequently sending the encrypted aggregated gradient, its signature, and the signatures of the online users' gradient hash values to all online users.

Auxiliary Server S_1: S_1 acts as the aggregator of the auxiliary ciphertext components. First, it interacts with S_0 to determine the online users' set U. Second, S_1 re-encrypts and aggregates the auxiliary ciphertext components received from the online users. Finally, S_1 sends the aggregated auxiliary ciphertext components to S_0.

The workflow is illustrated in Fig. 2.

4.3. Secure objectives

The security objectives of our scheme are to achieve the four properties:

(1) Correctness: If all parties correctly execute the protocol, the correct aggregated gradient will always be obtained.
(2) Privacy: In the scenario where two servers do not collude and at least two users remain uncorrupted, no one can obtain the local gradient of any single honest user.
(3) Dropout-tolerance: Even if some users drop out, the scheme can still correctly aggregate the gradients of the remaining users who have not dropped out.
(4) Verifiability: The gradient that is verified and accepted by users must be correct.

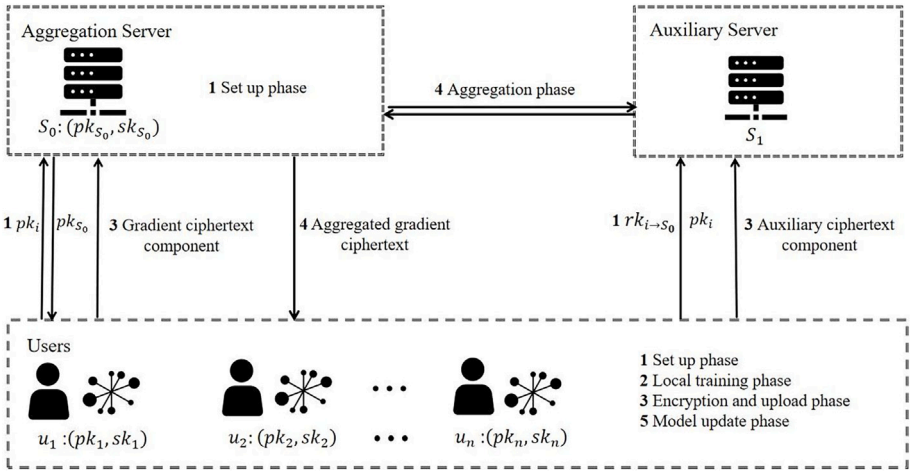


Fig. 2. System workflow.

4.4. Security model

The non-colluding dual/multi-server architecture [10,13,23] is a common model consisting of two or more semi-trusted servers. The non-colluding assumption is strong, but it reduces the trust requirements for the servers, allowing them to be semi-trusted. If multiple servers are allowed to collude, they essentially become a single server. To ensure privacy, it is typically required that a single server be fully trusted, or that additional protective measures be implemented. In practical deployments, the risk of collusion can be minimized by having two servers operated by independent entities with no incentive to cooperate, such as different cloud service providers or departments with conflicting interests. The non-colluding semi-trusted dual-server model is generally easier to achieve compared to a fully trusted single-server model.

Under the non-colluding dual-server model, privacy-preserving federated learning scheme faces two types of adversaries:

(1) **Type *I* Adversary $\mathcal{A}_1$**: A curious auxiliary server may collude with up to $n-2$ corrupted users to try to obtain the local model gradients of honest (uncorrupted) users. Specifically, $\mathcal{A}_1$ can gain access to all re-encryption keys and the secret keys of the corrupted users, but cannot obtain the secret key of the aggregation server.
(2) **Type *II* Adversary $\mathcal{A}_2$**: A curious aggregation server may collude with up to $n-2$ corrupted users to try to obtain the local model gradients of honest (uncorrupted) users. Specifically, $\mathcal{A}_2$ can gain access to the secret keys of the corrupted users and the aggregation server, but cannot obtain all re-encryption keys and the secret keys of the honest users.

The privacy requirements state that neither type of adversary should obtain any information about the local model gradients of honest users. Referring to the definitions provided by Joye et al. [26], we present the formal games and definitions.

GAME 1 ($\mathcal{A}_1$):

(1) **Initialization**: The challenger C initializes the system, generates the system public parameters and the public keys for the aggregation server and the users, and then sends them to $\mathcal{A}_1$.
(2) **Query**:

- **User Secret Key Queries**: $\mathcal{A}_1$ queries the secret key of u_i. If u_i is corrupted, return the corresponding secret key. Otherwise, reject the request.
- **Re – encryption Key Queries**: $\mathcal{A}_1$ queries the re-encryption key of u_i. C return the corresponding key.

(3) **Challenge** $\mathcal{A}_1$ selects and sends a set of honest users S^* and two sets of model gradient plaintexts $\{w_i^0\}_{i\in S^*}$, $\{w_i^1\}_{i\in S^*}$. C chooses $\beta \in_R \{0,1\}$, generates the ciphertext C_i of w_i^β for each user $u_i \in S^*$, and returns $\{C_i\}_{i\in S^*}$ to $\mathcal{A}_1$.
(4) **Guess**: $\mathcal{A}_1$ outputs the guess $\beta' \in \{0,1\}$.
The advantage of $\mathcal{A}_1$ is defined as:

$$Adv^{GAME1}(\mathcal{A}_1) = \left| Pr[\beta = \beta'] - \frac{1}{2} \right|.$$

GAME 2 ($\mathcal{A}_2$):

(1) **Initialization**: Same as in **GAME 1**.
(2) **Query**:

- **User Secret Key Queries**: Same as **GAME 1**.
- **Aggregation Server Key Queries**: $\mathcal{A}_2$ queries the secret key of the aggregation server. C return the corresponding key.

(3) **Challenge** $\mathcal{A}_2$ selects a set of honest users S^* and two sets of model gradient plaintexts $\{w_i^0\}_{i\in S^*}, \{w_i^1\}_{i\in S^*}$ for the challenge. C chooses $\beta \in_R \{0,1\}$, generates the ciphertext C_i of w_i^β for each user $u_i \in S^*$, and returns $\{C_i\}_{i\in S^*}$ to $\mathcal{A}_2$.

(4) **Guess**: $\mathcal{A}_2$ ouputs his/her guess $\beta' \in \{0,1\}$.
The advantage of $\mathcal{A}_2$ is defined as:

$$Adv^{GAME2}(\mathcal{A}_2) = \left| Pr[\beta = \beta'] - \frac{1}{2} \right|.$$

Definition 1. A PPFL scheme achieves privacy if the advantage $Adv^{GAME1}(\mathcal{A}_1)$ and $Adv^{GAME2}(\mathcal{A}_2)$ are both negligible for any PPT adversary $\mathcal{A}_1$ and $\mathcal{A}_2$ in **GAME 1** and **GAME 2**.

4.5. Scheme description

The basic notation used in the scheme is provided in Table 1. For simplicity, we assume that the gradient vectors are vectors in $\mathbb{Z}_p^d$, as it is easy to find a one-to-one mapping between gradient vectors and vectors in $\mathbb{Z}_p^d$. The proposed scheme is as follows:

1. **Initialization Phase**:
   (1) System Parameter Generation: S_0 generates system parameters and guides Federated Learning.
   - Input a security parameter λ, choose a prime number p of length λ, generate two multiplicative cyclic groups $\mathbb{G}$, $\mathbb{G}_T$ with the same order p and a bilinear map $e : \mathbb{G}\times\mathbb{G} \to \mathbb{G}_T$. Choose a generator g of group $\mathbb{G}$, compute $Z = e(g,g)$.
   - Choose a secure hash function $H_{\mathbb{G}} : \{0,1\}^* \to \mathbb{G}$.
   - Choose d different elements $g_1, g_2, \ldots, g_d$ in $\mathbb{G}$.
   - Choose d pairwise coprime positive integers $m_1, m_2, \ldots, m_d$.
   - Output the system public parameters:

$$pp = (p, g, \mathbb{G}, \mathbb{G}_T, e, Z, H_{\mathbb{G}}, g_1, g_2, \ldots, g_d, m_1, m_2, \ldots, m_d).$$

   (2) S_0 initializes the global gradient vector $\boldsymbol{w}_0$ and sends it to all users.
   (3) Key Generation: Inputting the system public parameters pp,
   - u_i chooses $v_i, s_i \in_R \mathbb{Z}_p$, sets $sk_i = v_i, ssk_i = s_i$, and computes $pk_i = Z^{v_i}, vpk_i = g^{s_i}$. u_i sends pk_i, vpk_i to S_0, S_1 and all users.
   - S_0 chooses $v_{S_0} \in_R \mathbb{Z}_p$, sets$sk_{S_0} = v_{S_0}$, computes $pk_{S_0} = g^{v_{S_0}}$, and sends pk_{S_0} to all users.
   (4) $Re-KeyGen$: Inputting the system public parameters pp, u_i uses sk_i and pk_{S_0} to compute $rk_{i\to S_0} = pk_{S_0}^{v_i} = g^{v_i v_{S_0}}$, and transmits $rk_{i\to S_0}$ through a secure channel to S_1.
2. **Local Training Phase**: In t-th round training, each user u_i uses the global gradient $\boldsymbol{w}_{t-1}$ and local dataset D_i to generate the local gradient $\boldsymbol{w}_{t,i}$. For the sake of brevity, the subscript t will be omitted in the following text, and $\boldsymbol{w}_i$ will be used instead of $\boldsymbol{w}_{t,i}$.

Table 1
Notations.

Notations	Description
u_i	i-th user.
S_i	S_0, S_1.
n	The number of users.
U_0, U_1	Online users sets received by S_0 and S_1.
U	Online user set $U = U_0 \cap U_1$.
pk_i, sk_i	Public and secret key of u_i.
pk_{S_0}, sk_{S_0}	Public and secret key of S_0.
$rk_{i\to S_0}$	Re-encryption key computed by u_i.
$\boldsymbol{w}_i$	Local gradient vector of u_i at t-th round.
$\boldsymbol{w}_{agg}$	Aggregated gradient vector at t-th round.
$\boldsymbol{w}_t$	Global gradient vector at t-th round.
$\hat{\boldsymbol{w}}_i$	The vector obtained by processing $\boldsymbol{w}_i$ using the CRT.
$\hat{\boldsymbol{w}}_{agg}$	The vector obtained by processing $\boldsymbol{w}_{agg}$ using the CRT.
C_{i1}^2	Second-level auxiliary ciphertext of u_i.
C_{i2}^2	Second-level gradient ciphertext of u_i.
$C_{iS_0}^1$	First-level ciphertext after re-encryption for C_{i1}^2.
$[\boldsymbol{w}_{agg}]_{pk_i}^1$	First-level ciphertext of $\boldsymbol{w}_{agg}$ under pk_i.
$H_{lhh}(\hat{\boldsymbol{w}}_i)$	Linear homomorphic hash function of $\hat{\boldsymbol{w}}_i$.
$H_{lhh}(\hat{\boldsymbol{w}}_{agg})$	Linear homomorphic hash function of $\hat{\boldsymbol{w}}_{agg}$.
$\sigma_{C_{i1}^2}, \sigma_{C_{i2}^2}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)}, \sigma_{S_0 i}$	Signatures of C_{i1}^2,C_{i2}^2,$H_{lhh}(\hat{\boldsymbol{w}}_i)$,$[\boldsymbol{w}_{agg}]_{pk_i}^1 \| U$.

3. **Encryption and Upload Phase**:

Each user u_i executes:

(1) Chooses $r_i \in_R \mathbb{Z}_p$, encrypts the local gradient $\boldsymbol{w}_i$ using pk_i, computes

$$\begin{aligned} C_{i1}^2 &= g^{r_i}, \\ C_{i2}^2 &= \left(Z^{w_i^1} pk_i^{r_i}, Z^{w_i^2} pk_i^{r_i}, \ldots, Z^{w_i^d} pk_i^{r_i} \right) \\ &= \left(Z^{w_i^1 + v_i r_i}, Z^{w_i^2 + v_i r_i}, \ldots, Z^{w_i^d + v_i r_i} \right). \end{aligned} \tag{16}$$

C_{i1}^2 is referred to as the auxiliary ciphertext component, and C_{i2}^2 is referred to as the gradient ciphertext component.

(2) To reduce computational overhead in high-dimensional scenarios, packs the local gradient vector $\boldsymbol{w}_i = (w_i^1, w_i^2, \ldots, w_i^d)$ into a lower-dimensional vector $\hat{\boldsymbol{w}}_i = \left(\hat{w}_i^1, \hat{w}_i^2, \ldots, \hat{w}_i^{\left\lceil \frac{d}{100} \right\rceil} \right)$ using the CRT.

(3) Computes the homomorphic hash value of $\hat{\boldsymbol{w}}_i$

$$H_{lhh}(\hat{\boldsymbol{w}}_i) = \prod_{l=1}^{\left\lceil \frac{d}{100} \right\rceil} g_l^{\hat{w}_i^l}. \tag{17}$$

(4) Signs C_{i1}^2,C_{i2}^2 and the homomorphic hash value $H_{lhh}(\hat{\boldsymbol{w}}_i)$ using ssk_i respectively.

$$\begin{aligned} \sigma_{C_{i1}^2} &= H_{\mathbb{G}}(C_{i1}^2)^{s_i}, \\ \sigma_{C_{i2}^2} &= H_{\mathbb{G}}(C_{i2}^2)^{s_i}, \\ \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)} &= H_{\mathbb{G}}(H_{lhh}(\hat{\boldsymbol{w}}_i)^{s_i}. \end{aligned} \tag{18}$$

(5) Sends $(C_{i2}^2, H_{lhh}(\hat{\boldsymbol{w}}_i), \sigma_{C_{i2}^2}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)})$ to S_0, and sends $(C_{i1}^2, \sigma_{C_{i1}^2})$ to S_1.

4 **Aggregation Phase**

(1) S_0 verifies and determines online users set U_0: After receiving $(C_{i2}^2, H_{lhh}(\hat{\boldsymbol{w}}_i), \sigma_{C_{i2}^2}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)})$, computes

$$\begin{aligned} h_{C_{i2}^2} &= H_{\mathbb{G}}(C_{i2}^2), \\ h_{H_{lhh}(\hat{\boldsymbol{w}}_i)} &= H_{\mathbb{G}}(H_{lhh}(\hat{\boldsymbol{w}}_i)), \end{aligned} \tag{19}$$

verifies whether the two equations hold:

$$\begin{aligned} e(vpk_i, h_{C_{i2}^2}) &= e(\sigma_{C_{i2}^2}, g), \\ e(vpk_i, h_{H_{lhh}(\hat{\boldsymbol{w}}_i)}) &= e(\sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)}, g). \end{aligned} \tag{20}$$

If both equations hold true, add u_i to the set U_0, where U_0 is initialized as an empty set. Sends U_0 to S_1.

(2) S_1 verifies and determines online users set U_1: After receiving $(C_{i1}^2, \sigma_{C_{i1}^2})$, computes

$$h_{C_{i1}^2} = H_{\mathbb{G}}(C_{i1}^2), \tag{21}$$

verifies whether the equation

$$e(vpk_i, h_{C_{i1}^2}) = e(\sigma_{C_{i1}^2}, g). \tag{22}$$

holds, if it is, adds u_i to the set U_1, where U_1 is initialized as an empty set.

(3) S_1 determines the online user set and performs re-encryption: Receiving the set U_0, S_1 computes the total set of online users $U = U_0 \cap U_1$. For each user $u_i \in U$, S_1 uses $rk_{i \to S_0}$ to re-encrypt C_{i1}^2 into

$$C_{iS_0}^1 = e(C_{i1}^2, rk_{i \to S_0}) = e(g^{r_i}, g^{v_i v_{S_0}}) = Z^{v_{S_0} v_i r_i}. \tag{23}$$

(4) S_1 aggregates the re-encrypted ciphertexts of online users, computes

$$C_{agg,1} = \prod_{i \in U} C_{iS_0}^1 = Z^{v_{S_0} \sum_{i \in U} v_i r_i} \tag{24}$$

and sends $C_{agg,1}$ and U to S_0.

(5) Receiving $C_{agg,1}$ and U, S_0 aggregates the encrypted gradients from all online users $u_i, i \in U$,

$$C_{agg,2} = \prod_{i \in U} C_{i2}^2, \tag{25}$$

and computes

$$Z^{\boldsymbol{w}_{agg}} = C_{agg,2} \cdot \left((C_{agg,1})^{(v_{S_0})^{-1}} \right)^{-1}, \tag{26}$$

furthermore computes the aggregated gradient $\boldsymbol{w}_{agg}$.

(6) S_0 packs the aggregated gradient vector $\boldsymbol{w}_{agg}$ into a lower-dimensional vector $\hat{\boldsymbol{w}}_{agg} = \left(\hat{w}^1_{agg}, \hat{w}^2_{agg}, \ldots, \hat{w}^{\left\lceil \frac{d}{100} \right\rceil}_{agg}\right)$ using the CRT.

(7) S_0 verifies whether the equation

$$H_{lhh}(\hat{\boldsymbol{w}}_{agg}) = \prod_{i \in U} H_{lhh}(\hat{\boldsymbol{w}}_i), \tag{27}$$

holds, if it is, S_0 chooses $r_{S_0} \in_R \mathbb{Z}_p$, and encrypts $\boldsymbol{w}_{agg}$ into $[\boldsymbol{w}_{agg}]^1_{pk_i}$ using pk_i for each online user u_i.

(8) S_0 signs $[\boldsymbol{w}_{agg}]^1_{pk_i}$ and U

$$\sigma_{S_0 i} = H_G\left(\left[\boldsymbol{w}_{agg}\right]^1_{pk_i} \parallel U\right)^{v_{S_0}}. \tag{28}$$

(9) S_0 sends $\left(\left[\boldsymbol{w}_{agg}\right]^1_{pk_i}, \sigma_{S_0 i}, \left\{H_{lhh}(\hat{\boldsymbol{w}}_j), \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_j)}\right\}_{j \in U}, U\right)$ to u_i.

5. **Verify and Update Phase**:

Receiving $\left(\left[\boldsymbol{w}_{agg}\right]^1_{pk_i}, \sigma_{S_0 i}, \left\{H_{lhh}(\hat{\boldsymbol{w}}_j), \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_j)}\right\}_{j \in U}, U\right)$, u_i executes:

(1) Computes $h_{S_0 i} = H_{\mathbb{G}}\left(\left[\boldsymbol{w}_{agg}\right]^1_{pk_i} \parallel U\right)$, and verifies whether the equation $e(pk_{S_0}, h_{S_0 i}) = e(\sigma_{S_0 i}, g)$ holds.

(2) If it does, decrypts $[\boldsymbol{w}_{agg}]^1_{pk_i}$ and obtains $\boldsymbol{w}_{agg}$.

(3) Packs the aggregated gradient vector $\boldsymbol{w}_{agg}$ into a lower-dimensional vector $\hat{\boldsymbol{w}}_{agg} = (\hat{w}^1_{agg}, \hat{w}^2_{agg}, \cdots, \hat{w}^{\left\lceil \frac{d}{100} \right\rceil}_{agg})$ using the CRT.

(4) Verifies whether the following equations hold:

$$\begin{aligned} H_{lhh}(\hat{\boldsymbol{w}}_{agg}) &= \prod_{i \in U} H_{lhh}(\hat{\boldsymbol{w}}_i), \\ e(vpk_j, H_G(H_{lhh}(\hat{\boldsymbol{w}}_j))) &= e(\sigma_{H_{lhh}(\hat{\boldsymbol{w}}_j)}, g). \end{aligned} \tag{29}$$

(5) If they do, computes $\boldsymbol{w}_t = \frac{1}{|U|}\boldsymbol{w}_{agg}$ as the initial global gradient for the next round.

5. Security analysis

We prove that the proposed scheme satisfies the four security properties: correctness, privacy, dropout-tolerance, and verifiability.

Theorem 1. *If each step of the scheme is executed correctly, then each user will receive the correct aggregation of the gradients of undropped users, which means that the scheme achieves both correctness and dropout-tolerance.*

Proof. The correctness of the BLS and LHH schemes ensures that all signatures and hash values can pass the verification. The correctness of the H-PRE ensures that the re-encryption and aggregation of the gradient ciphertexts are correct, and the decryption of the aggregated gradient ciphertext is also correct.

Since the data uploaded by users are independent of each other, the users who have dropped out do not affect the aggregation of gradients from the online users. The scheme only aggregates the gradients of online users whose data have been received by both servers, ensuring that the data are complete. According to the correctness, the scheme correctly aggregates the gradients of users who have not dropped out. □

Lemma 1. *Under the DDH-v1 assumption, any PPT Type-I adversary has a negligible advantage in attacking our scheme.*

Proof. Assume that a Type-I adversary $\mathcal{A}_1$ can attack our scheme with an advantage ϵ. We construct the following algorithm $\mathcal{B}$, which leverages $\mathcal{A}_1$'s attacking capability to solve the DDH-v1 problem. For the sake of a more concise presentation, without loss of generality, the proof is presented using dimension $d = 1$. □

Given a random instance (Y, T) of the DDH-v1 problem, $\mathcal{B}$ interacts with $\mathcal{A}_1$ as follows:

(1) **Initialization**

- After receiving $(\mathbb{G}, \mathbb{G}_T, p, e, g, e(g,g)^a, g^b, g^{ac})$, generates the other system parameters according to the initialization algorithm of the scheme, and sends the system public parameters

$$pp = (p, g, \mathbb{G}, \mathbb{G}_T, e, Z, H_{\mathbb{G}}, g_1, g_2, \ldots, g_d, m_1, m_2, \ldots, m_d).$$

  to $\mathcal{A}_1$.
- Chooses $v_i, s_i \in_R \mathbb{Z}_p$. If u_i is an honest user, computes $pk_i = (e(g,g)^a)^{v_i}$ and $vpk_i = g^{s_i}$. If u_i is a corrupted user, let $sk_i = v_i$, compute $pk_i = Z^{v_i}$ and the verification public key $vpk_i = g^{s_i}$. Adds (pk_i, sk_i) to the key list and sends the public keys to $\mathcal{A}_1$.
- Sets the aggregation server S_0's public key be g^c, and sends it to $\mathcal{A}_1$.

(2) **Query**:

- **User Secret Key Queries**: $\mathcal{A}_1$ queries the secret key of u_i. If u_i is honest, it rejects the request. Otherwise, C finds the corresponding secret key from the key list and sends it to $\mathcal{A}_1$.
- **Re – encryption Key Queries**: $\mathcal{A}_1$ queries the re-encryption key of u_i. If u_i is honest, computes $rk_{i\to S_0} = (g^{ac})^{v_i}$. Otherwise, computes $rk_{i\to S_0} = (g^c)^{v_i}$ and returns $rk_{i\to S_0}$ to $\mathcal{A}_1$

(3) **Challenge**: $\mathcal{A}_1$ selects and sends a set of honest users S^* and two sets of model gradient plaintexts $\{w_i^0\}_{i\in S^*}, \{w_i^1\}_{i\in S^*}$. C chooses $\beta \in_R \{0,1\}$, chooses $r_i \in_R \mathbb{Z}_p$ for each user $u_i \in S^*$, computes:

$$C_{i1}^2 = (g^b)^{r_i}, C_{i2}^2 = Z^{w_i^\beta} T^{r_i}, H_{lhh}(\hat{\boldsymbol{w}}_i) = \prod_{l=1}^{\left\lceil \frac{d}{100} \right\rceil} g_l^{\hat{w}_i^l}.$$

$$\sigma_{C_{i1}^2} = H_{\mathbb{G}}(C_{i1}^2)^{s_i}, \sigma_{C_{i2}^2} = H_{\mathbb{G}}(C_{i2}^2)^{s_i}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)} = H_{\mathbb{G}}(H_{lhh}(\hat{\boldsymbol{w}}_i)^{s_i}, \tag{30}$$

and sends $\left\{C_{i1}^2, C_{i2}^2, H_{lhh}(\hat{\boldsymbol{w}}_i), \sigma_{C_{i1}^2}, \sigma_{C_{i2}^2}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)}\right\}_{i\in S^*}$ to $\mathcal{A}_1$.

(4) **Guess**: $\mathcal{A}_1$ ouputs his/her guess $\beta' \in \{0,1\}$. If $\beta = \beta'$, $\mathcal{B}$ outputs 1, indicating $T = D$. Otherwise, $\mathcal{B}$ outputs 0, indicating $T = R$.

Advantage Analysis

- When $T = D$, (C_{i1}^2, C_{i2}^2) are the correct ciphertexts obtained by encrypting w_i^β using the random number br. In this case, the simulation of $\mathcal{B}$ is perfect, thus allowing $\mathcal{A}_1$ to exploit its attack advantage. According to the output rule of $\mathcal{B}$,

$$Pr[\mathcal{B}(Y,T) = 1 \mid T = D] = Pr[\beta = \beta'] = \frac{1}{2} \pm \epsilon.$$

- When $T = R$, assuming $T = R = e(g,g)^x$,(C_{i1}^2, C_{i2}^2) can be rewritten as

$$C_{i1}^2 = (g^b)^{r_i},$$

$$C_{i2}^2 = Z^{w_i^\beta} T^{r_i} = e(g,g)^{w_i^\beta + (x-ab)r_i} e(g,g)^{abr_i}. \tag{31}$$

(C_{i1}^2, C_{i2}^2) is the ciphertext of the random number $w_i^\beta + (x-ab)r_i$, independent of β. Therefore, the advantage of $\mathcal{A}_1$ is 0. Hence,

$$Pr[\mathcal{B}(Y,T) = 1 \mid T = R] = \frac{1}{2}.$$

We have

$$|Pr[\mathcal{B}(Y,T) = 1 \mid T = D] - Pr[\mathcal{B}(Y,T) = 1 \mid T = R]| = \left|\frac{1}{2} \pm \epsilon - \frac{1}{2}\right| = \epsilon.$$

The above equation indicates that if $\mathcal{A}_1$ can attack our scheme with an advantage ϵ, then $\mathcal{B}$ can solve the DDH-v1 problem with the same advantage ϵ. Therefore, under the DDH-v1 assumption, any PPT Type-I adversary's advantage in attacking our scheme is negligible.

Lemma 2. *Under the DDH-v2 assumption, any PPT Type-Π adversary has a negligible advantage in attacking our scheme.*

Proof. Assume that a Type-Π adversary $\mathcal{A}_2$ can attack our scheme with an advantage ϵ. We construct the following algorithm $\mathcal{B}$, which leverages $\mathcal{A}_2$'s attacking capability to solve the DDH-v2 problem. For the sake of a more concise presentation, without loss of generality, the proof is presented using dimension $d = 1$.

Given a random instance (Y,T) of the DDH-v2 problem, $\mathcal{B}$ interacts with $\mathcal{A}_2$:

(1) **Initialization**

- Same as the initialization in Lemma 1.
- Chooses $v_{S_0} \in_R \mathbb{Z}_p$, sets $sk_{S_0} = v_{S_0}$, computes $pk_{S_0} = g^{v_{S_0}}$, and sends pk_{S_0} to $\mathcal{A}_2$.

(2) **Query**:

- **User Secret Key Queries**: Same as the user secret key queries in Lemma 1.
- **Aggregation Server S_0 Secret Key Queries**: $\mathcal{A}_2$ queries the secret key of the aggregation server S_0. C returns the corresponding key v_{S_0} to $\mathcal{A}_2$.

(3) **Challenge** $\mathcal{A}_2$ selects and sends a set of honest users S^* and two sets of model gradient plaintexts $\{w_i^0\}_{i\in S^*}, \{w_i^1\}_{i\in S^*}$. C chooses $\beta \in_R \{0,1\}$, chooses $r_i \in_R \mathbb{Z}_p$ for each user $u_i \in S^*$, computes

$$C_{i1}^2 = (g^b)^{r_i}, C_{i2}^2 = Z^{w_i^\beta} T^{r_i}, H_{lhh}(\hat{\boldsymbol{w}}_i) = \prod_{l=1}^{\left\lceil \frac{d}{100} \right\rceil} g_l^{\hat{w}_i^l}.$$

$$\sigma_{C_{i1}^2} = H_{\mathbb{G}}(C_{i1}^2)^{s_i}, \sigma_{C_{i2}^2} = H_{\mathbb{G}}(C_{i2}^2)^{s_i}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)} = H_{\mathbb{G}}(H_{lhh}(\hat{\boldsymbol{w}}_i)^{s_i}, \tag{32}$$

and sends $\left\{ C_{i1}^2, C_{i2}^2, H_{lhh}(\hat{\boldsymbol{w}}_i), \sigma_{C_{i1}^2}, \sigma_{C_{i2}^2}, \sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)} \right\}_{i\in S^*}$ to $\mathcal{A}_2$.

(4) **Guess**: $\mathcal{A}_2$ outputs his/her guess $\beta' \in \{0,1\}$. If $\beta = \beta'$, $\mathcal{B}$ outputs 1, indicating $T = D$. Otherwise, $\mathcal{B}$ outputs 0, indicating $T = R$.

Similar analysis to Lemma 1 shows

$$|Pr[\mathcal{B}(Y,T) = 1 \mid T = D] - Pr[\mathcal{B}(Y,T) = 1 \mid T = R]| = \left| \frac{1}{2} \pm \epsilon - \frac{1}{2} \right| = \epsilon.$$

The above equation indicates that if $\mathcal{A}_2$ can attack our scheme with an advantage ϵ, then $\mathcal{B}$ can solve the DDH-v2 problem with the same advantage ϵ. Therefore, under the DDH-v2 assumption, any PPT Type-Π adversary's advantage in attacking our scheme is negligible.

According to Lemmas 1 and 2, we have

Theorem 2. *Under the DDH-v1 and DDH-v2 assumptions, our scheme achieves privacy.*

Theorem 3. *Our scheme achieves verifiability.*

Proof. Each $H_{lhh}(\hat{\boldsymbol{w}}_i)$ in $\prod_{i\in U} H_{lhh}(\hat{\boldsymbol{w}}_i)$ has a signature $\sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)}$ from the corresponding u_i to ensure its integrity. The unforgeability of the BLS signature scheme guarantees that as long as the equation $e(vpk_i, h_{H_{lhh}(\hat{\boldsymbol{w}}_i)}) = e(\sigma_{H_{lhh}(\hat{\boldsymbol{w}}_i)}, g)$ holds, $H_{lhh}(\hat{\boldsymbol{w}}_i)$ is correct and has not been tampered with. □

The correctness of Theorem 1 ensures that the correct aggregated value of $\{\boldsymbol{w}_i\}_{i\in U}$ is $\boldsymbol{w}_{agg}$, and the correct aggregated value$\boldsymbol{w}_{agg}$, after being packed using the Chinese Remainder Theorem, results in $\hat{\boldsymbol{w}}_{agg}$, which satisfies $H_{lhh}(\hat{\boldsymbol{w}}_{agg}) = \prod_{i\in U} H_{lhh}(\hat{\boldsymbol{w}}_i)$. If an adversary can find a $\hat{\boldsymbol{w}}_{fate} \neq \hat{\boldsymbol{w}}_{agg}$ such that $H_{lhh}(\hat{\boldsymbol{w}}_{fate}) = \prod_{i\in U} H_{lhh}(\hat{\boldsymbol{w}}_i)$ holds, then this means that we have found a collision pair for the LHH. This contradicts the collision resistance of the LHH. Thus, the aggregated gradients that pass verification and are accepted by users are guaranteed to be correct, thereby ensuring the scheme's verifiability.

6. Experiment

We conduct experiments to analyze the proposed scheme and compare it with four existing similar schemes [10,13–15]. The experimental configuration includes a 64-bit Windows 10 operating system, an Intel(R) Core(TM) i5-10300 H CPU with a clock frequency of 2.50 GHz, and 16 GB of memory. The experiments are conducted using the Java Pairing-Based Cryptography (JPBC) library available at https://sourceforge.net/projects/ipbc/. The cryptographic constructions utilize a Type A pairing defined over the elliptic curve $y^2 = x^3 + x$ in the finite field $\mathbb{F}_q$, where the security parameter λ is set to 1024, and the field modulus q is a 1024-bit prime satisfying the congruence $q \equiv 3 \ mod \ 4$. The curve configuration provides standard security guarantees and supports efficient bilinear pairing operations. The gradient dimension ranges from 2000–10,000 and number of users is 100.

6.1. Computational overhead

Since the existing four schemes don't have verification mechanisms, the following comparison will be divided into categories with and without verification. Fig. 3 shows that the total computational overheads of all schemes increase linearly with the gradient dimension d. For simplicity, the term overhead refers to computational overhead throughout the section.

The overheads during the encryption phase for the five schemes are similar. The overheads increase linearly with the gradient dimension d. When $d = 2000$, it is approximately 2.73s, and when $d = 10000$, it increases to approximately 12.21s. Our scheme requires $d + 2$ exponentiation operations for encryption without verification, which is almost the same as the overhead of [10] and [13], while being less than the $d + 5$ power exponentiation operations of [14] and [15]. The overheads for the verification part are very low, approximately 0.37s.

Without considering the overhead for the verification part, the decryption overheads for the five schemes are fixed and independent of the dimension d. The overhead of our scheme is similar to that of [10] and slightly less than that of the other three schemes, approximately 0.02s. [13–15] have overheads of 0.3s, 0.5s and 0.5s, respectively.

The simulation results for servers' overheads are shown in Fig. 4. The overheads of the servers for all five schemes increase linearly with the gradient dimension d. Li et al. [10] have a higher overhead, while the other four schemes have very low overheads. For example, when $d = 6000$, the aggregation server's overhead of our scheme (excluding the verification part) is 7.16s. The overheads of [10,13–15] are 7.84s, 7.92s, 8.16s and 14.53s, respectively. Our scheme has the lowest aggregation server overheads without verification, which is 49 % of the overheads of [10].

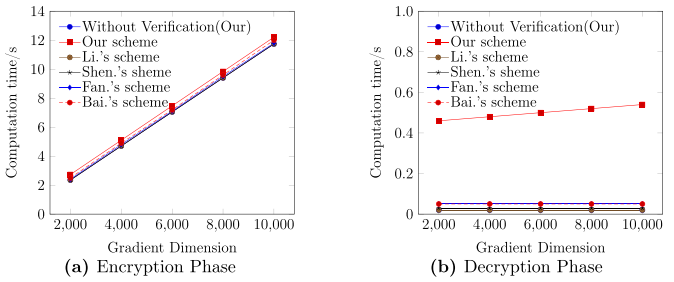


Fig. 3. Comparison of computational overhead. (a)Encryption phase; (b)Decryption phase.

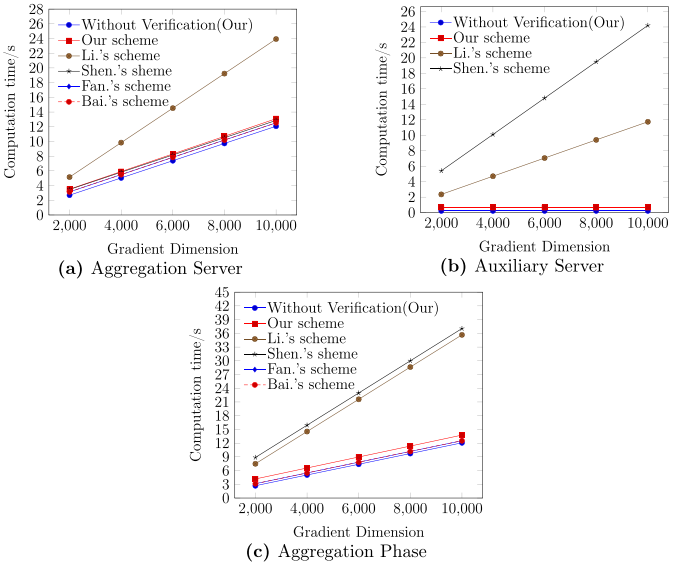


Fig. 4. Comparison of computational overhead for servers. (a)Aggregation server. (b)Auxiliary server. (c)Aggregation phase. (d)Total computational overhead.

The overhead of the auxiliary server in our scheme is independent of the gradient dimension and remains very low, only 0.66s on average, which is 2 % and 4 % of the overheads of [13] and [10], respectively. For example, when $d = 6000$, Li et al. [10] have the middle overheads with an average of 7.05s. [13] has the largest computational overhead with an average of 14.79s. Fan et al. [14] and [15] do not have an auxiliary server.

The total overheads of the aggregation phase for all five schemes increase linearly with the gradient dimension d. Our scheme, Fan et al. [14] and [15] have less overhead. When $d = 2000$, it is approximately 4.18s, and when $d = 10000$, it increases to approximately 13.76s. The verification overhead of our scheme during the aggregation phase averages 1.58s. The overhead of our scheme in the aggregation phase is the lowest without verification, which is 40 % and 44 % of that of [10,13], respectively.

The simulation results of the total overheads are shown in Fig. 5. The total overheads of all five schemes increase linearly with the gradient dimension d. Our scheme, Fan et al. [14] and [15] have fewer overheads. The total overhead of our scheme without verification is 45 % and 51 % of the overheads of [13] and [10], respectively. Our verification overhead averages 2.5s. Our scheme

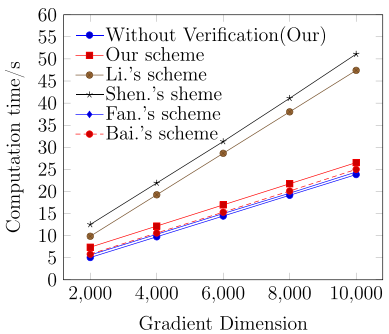


Fig. 5. Total computational overheads.

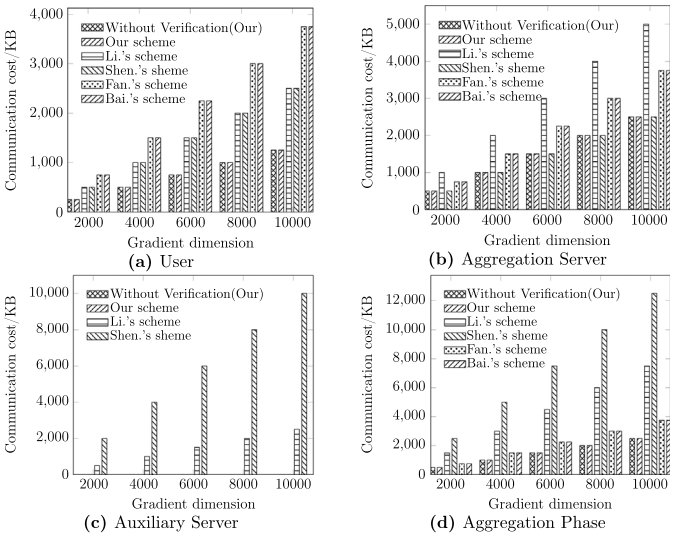


Fig. 6. Comparison of communication overhead. (a)User. (b)Aggregation server. (c)Auxiliary server. (d)Aggregation phase.

provides verifiability at a cost of 2.5s. Our simulation experiments are conducted on a personal laptop. If professional machine learning training servers were used, the verification overhead would be much lower.

In summary, the overheads for all entities in the five schemes are on the order of seconds, making them practical for machine learning systems. Our scheme achieves verifiability with a very low overhead.

6.2. Communication overhead

The communication overheads are shown in Fig. 6. For simplicity, the term overhead refers to communication overhead throughout the section. The results show that the overheads of all schemes increase linearly with the gradient dimension d.

The overheads for users are shown in Fig. 6a. A single user's overhead in our scheme is the lowest. When $d = 2000$, it is approximately 250.63KB, and when $d = 10000$, it increases to approximately 1250.63KB. The overhead for the verification part is small, around 0.63KB. A single user's overhead in our scheme without verification is 33 %, 33 %, 50 %, and 50 % of that in [10,13–15], respectively.

The aggregation server's overhead in our scheme is similar to that of [13]. When $d = 2000$, it is approximately 500.38KB, and when $d = 10000$, it increases to approximately 2500.38KB. The overhead for the verification part is small, around 0.38KB. The user's overhead in our scheme is 50 %, 67 %, and 67 % of that in [10,14,15] respectively.

The auxiliary server's overhead in our scheme is independent of the gradient dimension and remains very low. For example, when $d = 6000$, the overhead of [10] is moderate with an overhead of 1500KB, and the overhead of [13] is the highest requiring 8000KB. The overhead of our scheme is 0.13KB. Fan et al. [14] and [15] do not use an auxiliary server.

In terms of total overhead during the aggregation phase, the overheads of our scheme, Fan et al. [14] and [15] are similar. When $d = 2000$, it is approximately 500.50KB, and when $d = 10000$, it increases to approximately 2500.50KB. The overhead of our scheme during the aggregation phase is 33 % of that of [13] and 50 % of that of [10], respectively. The verification data, which can be either a digital signature or a hash value, has a very small size and requires only 0.5KB of overhead to achieve verifiability.

6.3. Accuracy

In ML, accuracy is one of the key metrics for evaluating model performance. We evaluate the performance of a Convolutional Neural Network (CNN) on two popular datasets, MNIST and CIFAR-10. The model consists of 4 convolutional layers and 1 max-pooling layer, with batch normalization and ReLU activation applied after each convolution. The intermediate features are flattened and then processed by a single fully connected layer to perform 10-class classification. The input images are first normalized to the range $[0, 1]$ before being fed into the network. The output of the network is a probability vector of length 10, representing the predicted probabilities for each class. Training is uniformly conducted using cross-entropy loss and the AdamW optimizer, with only the batch size ($BS = 16, 32, 64, 128, 256$) being adjusted. Each configuration is run for 20 rounds to quantify the impact of different settings on convergence speed and final accuracy. We update the model parameters on the training set and validate the accuracy after each epoch on the test set to evaluate model performance. The entire experiment starts with randomly initialized weights and does not rely on any pre-trained models.

Fig. 7 shows the classification accuracy of the model on the MNIST and CIFAR-10 datasets. In Fig. 7a, the model accuracy gradually increases as the number of training rounds increases on the MNIST. The CNN model achieved an accuracy of 98 % by the 5th epoch. However, as training continued, the accuracy began to fluctuate, indicating some degree of overfitting. Ultimately, when Round = 18,

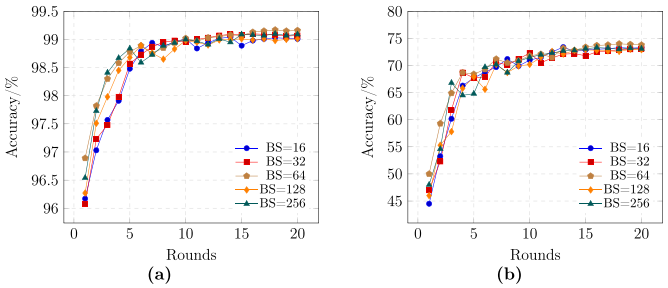


Fig. 7. Comparison of model accuracy under different batch sizes.(a)MNIST. (b)CIFAR-10.

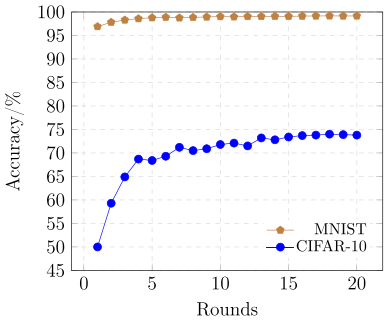


Fig. 8. Comparison of model accuracy for different datasets under the same batch size.

BS = 64, the model achieved its best performance with an accuracy of 99.2 %. In Fig. 7b, the model accuracy on the CIFAR-10 also gradually increases as the number of training rounds increases. When BS = 64, the model achieves the best classification performance with an accuracy of 74 %. Combining the experimental results from both datasets, we observe that Batch Size = 64 yields the optimal performance in terms of both accuracy and convergence.

In Fig. 8, under the condition of a fixed batch size of 64, we conducted a systematic evaluation of model performance using two datasets. The experimental results demonstrate that different model architectures exhibit significant differences in classification accuracy: specifically, the Convolutional Neural Network (CNN) achieved an accuracy of approximately 74 % on the CIFAR-10, while the accuracy on the MNIST exceeds 99.2 %. This difference primarily stems from the differences in feature complexity and data composition between the two datasets. The images in the CIFAR-10 dataset contain rich visual information and complex feature structures. The dataset is diverse and large-scale, posing higher demands on the model's analysis and recognition capabilities. In contrast, the MNIST dataset consists solely of grayscale images of handwritten Arabic numerals, with relatively simple and uniform features, making it easier for the model to achieve high classification performance.

7. Conclusion

Privacy-preserving federated learning (PPFL) is one of the research hotspots in ML and a popular topic at the intersection of information security and artificial intelligence. We propose a verifiable PPFL scheme based on H-PRE. The scheme requires only that the server be semi-trusted, and tolerates user dropout, achieving correctness, privacy, and verifiability in federated learning. It demonstrates significant computational and communication cost advantages, making it highly practical and promising for real-world applications. The main limitation of our scheme is the assumption that the two servers do not collude. In practical applications, if two servers collude, it will completely compromise the privacy of the scheme. Therefore, removing this assumption is a challenging task for future work. Achieving PPFL based on H-PRE in a semi-trusted single-server model or a multi-server model allowing collusion is one potential direction for future research.

CRediT authorship contribution statement

Zeyu Song: Writing – review & editing, Writing – original draft, Validation, Methodology, Investigation. **Zhenjie Huang:** Writing – review & editing, Supervision, Project administration, Methodology, Investigation, Funding acquisition. **Yiping Cai:** Investigation.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgement

This work was supported by the National Social Science Fund of China under Grant No. 21XTQ015.

Data availability

No data was used for the research described in the article.

References

[1] M.I. Jordan, T.M. Mitchell, Machine learning: trends, perspectives, and prospects, Science 349 (6245) (2015) 255–260.
[2] H.B. McMahan, E. Moore, D. Ramage, S. Hampson, B. Agüera y Arcas, Communication-efficient learning of deep networks from decentralized data, arXiv e-prints arXiv:1602.05629, 2016.
[3] D. Wang, S. Shi, Y. Zhu, Z. Han, Federated Analytics: opportunities and challenges, IEEE Netw. 36 (1) (2021) 151–158.
[4] V. Mothukuri, R.M. Parizi, S. Pouriyeh, Y. Huang, A. Dehghantanha, G. Srivastava, A survey on security and privacy of federated learning, Futur. Gener. Comput. Syst. 115 (2021) 619–640.
[5] R. Xu, B. Li, C. Li, J.B.D. Joshi, S. Ma, J. Li, TAPFed: threshold secure aggregation for privacy-preserving federated learning, IEEE Trans. Dependable Secure Comput. 21 (5) (2024) 4309–4323.
[6] J. Song, W. Wang, T.R. Gadekallu, J. Cao, Y. Liu, EPPDA: an efficient privacy-preserving data aggregation federated learning scheme, IEEE Trans. Netw. Sci. Eng. 10 (5) (2022) 3047–3057.
[7] G. Xu, H. Li, S. Liu, K. Yang, X. Lin, VerifyNet: secure and verifiable federated learning, IEEE Trans. Inf. Forensics Secur. 15 (2019) 911–926.
[8] P. Zhao, Z. Cao, J. Jiang, F. Gao, Practical private aggregation in federated learning against inference attack, IEEE Internet Things J. 10 (1) (2022) 318–329.
[9] Z. Shi, Z. Yang, A. Hassan, F. Li, X. Ding, A privacy preserving federated learning scheme using homomorphic encryption and secret sharing, Telecommun. Syst. 82 (3) (2023) 419–433.
[10] Y. Li, J. Lai, R. Zhang, M. Sun, Secure and efficient multi-key aggregation for federated learning, Inf. Sci. 654 (2024) 119830.
[11] N. Rodríguez-Barroso, D. Jiménez-López, M.V. Luzón, F. Herrera, E. Martínez-Cámara, Survey on federated learning threats: concepts, taxonomy on attacks and defences, experimental study and challenges, Inf. Fusion 90 (2023) 148–173.
[12] R. Wang, X. Yuan, Z. Yang, Y. Wan, M. Luo, D. Wu, RFLPV: a robust federated learning scheme with privacy preservation and verifiable aggregation in IoMT, Inf. Fusion 102 (2024) 102029.
[13] X. Shen, X. Luo, B. Wang, Y. Chen, D. Tang, L. Gao, et al., Privacy-preserving multi-party deep learning based on homomorphic proxy re-encryption, J. Syst. Archit. 144 (2023) 102983.
[14] C.-I. Fan, Y.-W. Hsu, C.-H. Shie, Y.-F. Tseng, Id-based multireceiver homomorphic proxy re-encryption in federated learning, ACM Trans. Sens. Netw. 18 (4) (2022) 1–25.
[15] Y. Bai, Y. Rao, H. Wu, J. Wang, W. Yang, G. Xing, J. Yang, X. Yuan, Using homomorphic proxy re-encryption to enhance security and privacy of federated learning-based intelligent connected vehicles, IET Inf. Secur. 2025 (1) (2025) 4632786.
[16] J. Ma, S.-A. Naas, S. Sigg, X. Lyu, Privacy-preserving federated learning based on multi-key homomorphic encryption, Int. J. Intell. Syst. 37 (9) (2022) 5880–5901.
[17] Y. Cai, W. Ding, Y. Xiao, Z. Yan, X. Liu, Z. Wan, SecFed: a secure and efficient federated learning based on multi-key homomorphic encryption, IEEE Trans. Dependable Secure Comput. 21 (4) (2023) 3817–3833.
[18] W. Du, M. Li, L. Wu, Y. Han, T. Zhou, X. Yang, A efficient and robust privacy-preserving framework for cross-device federated learning, Complex Intell. Syst. 9 (5) (2023) 4923–4937.
[19] J. Zhao, H. Zhu, F. Wang, R. Lu, H. Li, J. Tu, J. Shen, CORK: a privacy-preserving and lossless federated learning scheme for deep neural network, Inf. Sci. 603 (2022) 190–209.
[20] X. Shen, X. Luo, F. Yuan, B. Wang, Y. Chen, D. Tang, L. Gao, Verifiable privacy-preserving federated learning under multiple encrypted keys, IEEE Internet Things J. 11 (2) (2023) 3430–3445.
[21] X. Guo, Z. Liu, J. Li, J. Gao, B. Hou, C. Dong, T. Baker, Veri FL: communication-efficient and fast verifiable aggregation for federated learning, IEEE Trans. Inf. Forensics Secur. 16 (2020) 1736–1751.
[22] Y. Ren, Y. Li, G. Feng, X. Zhang, Privacy-enhanced and verification-traceable aggregation for federated learning, IEEE Internet Things J. 9 (24) (2022) 24933–24948.
[23] T. Eltaras, F. Sabry, W. Labda, K. Alzoubi, Q. Ahmedeltaras, Efficient verifiable protocol for privacy-preserving aggregation in federated learning, IEEE Trans. Inf. Forensics Secur. 18 (2023) 2977–2990.
[24] G. Ateniese, K. Fu, M. Green, S. Hohenberger, Improved proxy re-encryption schemes with applications to secure distributed storage, ACM Trans. Inf. Syst. Secur. 9 (1) (2006) 1–30.
[25] H. Shafagh, A. Hithnawi, L. Burkhalter, P. Fischli, S. Duquennoy, Secure sharing of partially homomorphic encrypted IOT data, in: Proceedings of the 15th ACM Conference on Embedded Network Sensor Systems, 2017, pp. 1–14.
[26] M. Joye, B. Libert, A scalable scheme for privacy-preserving aggregation of time-series data, in: International Conference on Financial Cryptography and Data Security, Springer, 2013, pp. 111–125.